

Benchmark Report: nbody

Five bodies, twenty thousand steps, and an interpreter between every calculation

Matan Cohen · Yuval Kogan | Technion | HWSW Final Project

A probe is falling toward Neptune, and somebody has to know where it will be in six hours.

Predicting motion means repeatedly adding up gravitational forces. This benchmark captures that computational core in miniature: the Sun and four gas giants, ten interacting pairs, and twenty thousand steps. The physics is compact; in Python, the machinery around each calculation is not. Our optimization asks how much of that machinery we can remove.

1. Workload and result

The Computer Language Benchmarks Game's nbody uses symplectic Euler integration under Newtonian gravity. Each timed iteration evaluates energy, calls `advance(0.01, 20000)`, then evaluates energy again. The stock kernel uses only the standard library, with no I/O in the timed region. State consists of five position/velocity/mass records (35 float values); the ten body pairs are constructed once and reused.

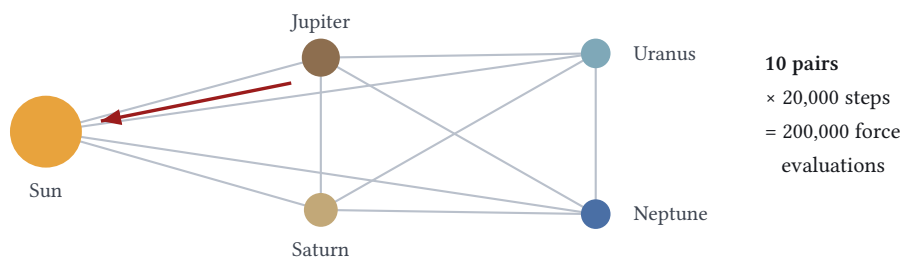


Figure 1: Ten fixed pairs × 20,000 steps = 200,000 pair-force evaluations.

Configuration	Mean ± SD	vs stock
Stock Python	231.20 ± 7.85 ms	1.00×
Optimized Python	143.13 ± 1.56 ms	1.62×

The optimization reduces runtime by **38.1%**, exceeding the course's 7% requirement. It specializes the interaction schedule into local-variable arithmetic while preserving operation order. Final state and energy compare exactly equal to stock after 20,000 steps. Energy is a useful physical sanity check; componentwise state comparison is the stronger equivalence check.

Measurement scope. Course QEMU/KVM VM, Ubuntu 22.04, release CPython 3.10.12, pyperformance 1.14.0; 120 measured values per configuration, every timed run pinned to guest CPU 0, measured from revision 2c8c754 through the documented runner scripts. Source: `results/vm_canonical_20260910_2c8c754/suite/{baseline,optimized}_nbody.json`, which also record the back end and CPU affinity (Appendix A7). SD denotes sample standard deviation, not a confidence interval. Profiling is separate from timing; the shared appendix records methods and provenance.

Worker processes account for much of the timing variation. Appendix A5 reports medians, interquartile ranges and uncertainty adjusted for worker clustering; the measured runtime reduction is much larger than the estimated uncertainty in either mean.

2. Profiling and optimization

Python-frame sampling places almost all benchmark work in `advance()`. The C-level debug-build profile identifies generic arithmetic dispatch, float allocation and list access as substantial costs. These debug-build percentages locate costs to investigate; release-build timing establishes the optimization's benefit.

Stock nbody, C-frame profile	Self	Inclusive
<code>_PyEval_EvalFrameDefault</code>	44.10%	99.31%
<code>binary_op1</code> (generic arithmetic dispatch)	4.90%	20.07%
<code>float_mul</code>	3.86%	3.86%
<code>PyFloat_FromDouble</code>	4.03%	5.82%
<code>float_dealloc</code>	3.48%	3.48%
<code>list_ass_item</code>	1.78%	1.78%
<code>PyNumber_AsSsize_t</code>	2.22%	4.87%
Float object handling, all symbols	16.25%	–
List access and index conversion, all symbols	11.26%	–

Self is time in the function itself; **inclusive** is time in it and everything it calls. The two grouped rows sum self time over `^(float_|PyFloat_)` and over `^(list_|listiter_|PyNumber_AsSsize_t|PyLong_AsSsize_t)` respectively.

The figure labels give total inclusive shares across a function's recorded frames; the outline identifies one representative frame. The flat perf table and the SVG may differ slightly through rounding and inline attribution (`list_ass_item`: 1.78% self here, 3.40% inclusive in the graph). Appendix A4/A6 records the extraction method.

Specialize the fixed schedule

The original loop revisits the same pairs and masses on every step. At import time, the optimized version emits an `advance()` function with the pair loop unrolled, coordinates in Python locals, and masses as literals. It loads state before the step loop and writes it back afterwards, so energy reporting still observes the original body objects.

This removes list subscripting from the generated step loop and avoids repeated pair unpacking. List operations remain at function entry and exit, amortized over 20,000 steps.

Before and after: the x update of one pair

The following excerpts abbreviate the shipped transformation. The y/z operations and other pairs are omitted only from the illustration. M0 and M1 denote the numeric mass literals that the generator inserts; they are not new runtime lookups.

Stock: inside the pair loop

```
for ([x1, y1, z1], v1, m1),
    ([x2, y2, z2], v2, m2) in pairs:
    dx = x1 - x2
    # dy, dz and mag as below
    b1m = m1 * mag
    b2m = m2 * mag
    v1[0] -= dx * b2m
    v2[0] += dx * b1m
```

Generated: pair (0, 1)

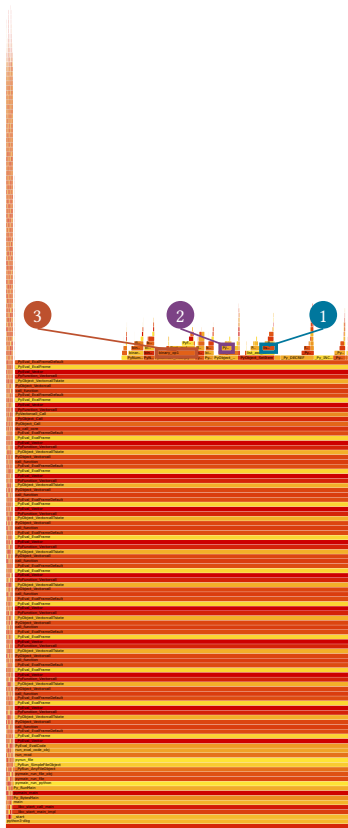
```
dx = x0 - x1
# dy, dz and mag as below
b1m = M0 * mag
b2m = M1 * mag
ux0 -= dx * b2m
ux1 += dx * b1m
```

Both compute `mag = dt * ((dx*dx + dy*dy + dz*dz) ** (-1.5))` in the same order. The optimization removes repeated pair unpacking and list reads/writes, not the force calculation. Coordinates and velocities enter locals once per invocation; after all steps, the generator writes them back to the original lists. These are still Python float objects, so removing indexing does not remove all interpreter arithmetic or allocation costs. Source: `benchmarks/bm_nbody/run_benchmark.py, _advance_source`.

Correctness constrains the arithmetic. The kernel retains `dt * (dsq ** -1.5)` and the stock pair-update order. A square-root/division replacement changes rounding and is not used in the exact software tier. The emitter accepts a body list and pair schedule, so the same transformation extends to larger systems.

Reading the profiles

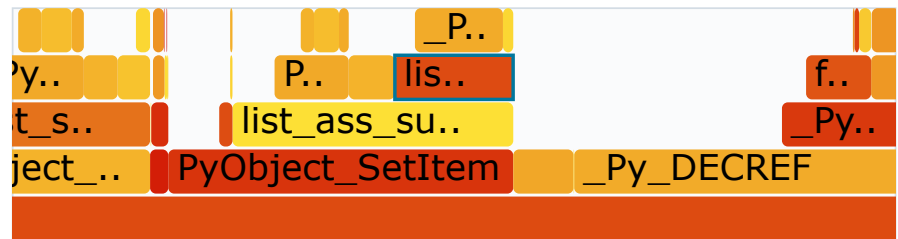
Full recorded flame graph



Highlighted regions - enlarged with surrounding frames

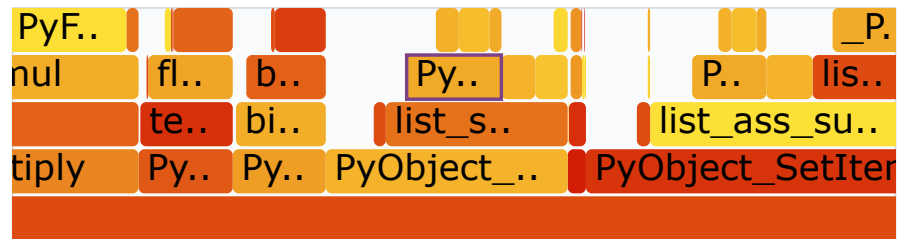
1. List stores

list_ass_item | 3.40% inclusive



2. Index conversion

PyNumber_AsSsize_t | 4.87% inclusive, across 2 per-line frames; widest 2.73% outlined



3. Generic arithmetic dispatch

binary_op1 | 20.09% inclusive, across 12 per-line frames; widest 10.55% outlined

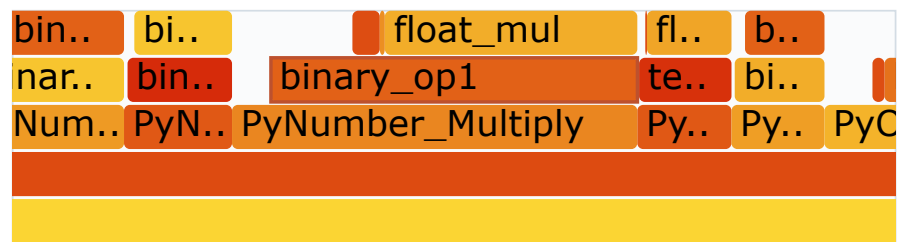
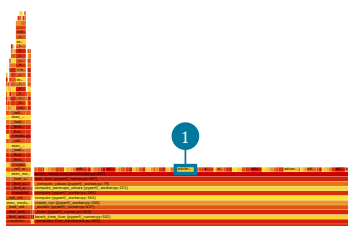


Figure 2: Full stock C-frame profile (debug CPython), with the same call paths outlined and enlarged. All original frames, widths and startup context are retained. Each label gives the function's total inclusive share; the outlined box is the widest of the per-line frames the profiler splits it into, and callers sit below it.

Full recorded flame graph



Highlighted regions - enlarged with surrounding frames

1. Generated integration loop

advance | 91.57% inclusive, across 95 per-line frames; widest 4.76% outlined

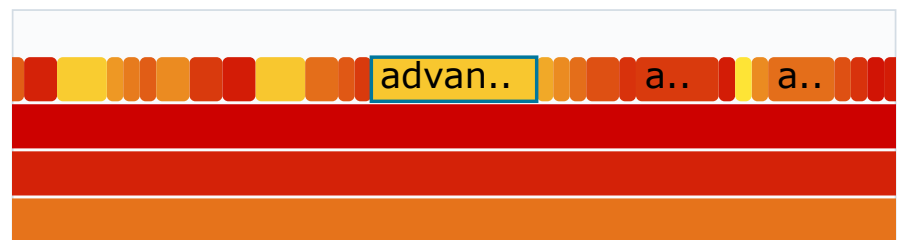


Figure 3: Full optimized Python-frame profile with the integration call path enlarged. Integration remains the hotspot. Independent normalization does not display the absolute runtime reduction from 231 to 143 ms.

3. Native execution: less work, not higher IPC

The Rust/PyO3 System executes all 20,000 steps in one call, with both energy evaluations also native. State and energy match stock exactly on the tested VM build; this does not guarantee equality across all compilers and math libraries.

Same benchmark file	Mean $\pm$ SD	vs Python
Python backend	145.30 $\pm$ 4.97 ms	1.00×
Native backend	9.530 $\pm$ 0.050 ms	15.25×

Source: canonical suite/{fallback,native}_nbody.json (Appendix A7). Both use direct pyperf; Section 1 uses pyperformance. Against canonical stock, native is 24.26× faster. Different Python denominators prevent multiplying the two reported ratios. Explicit backend selection and worker metadata establish what ran; native mode fails if unavailable.

Matched-work CPU counters

A separate pinned warm-loop experiment repeats the work 64 times per run. Medians of three runs are normalized per iteration, excluding setup/warmup (Appendix A2).

Metric per iteration	Python	Native
Elapsed time	141.08 ms	9.462 ms
Instructions	1,103.62 M	41.32 M
Cycles	336.24 M	22.55 M
IPC	3.28	1.83
Branches	185.97 M	3.92 M
Branch misses (rate)	583,560 (0.314%)	60 (0.00153%)
Generic cache references	3,006	255
Generic cache misses (rate)	59.6 (2.245%)	2.5 (1.134%)

Native needs 26.7× fewer instructions and 14.9× fewer cycles. Removing Python dispatch and object handling reduces work. Lower IPC still yields faster execution because so many fewer instructions remain; IPC alone does not measure useful work.

Cache misses are sparse and variable (Python 59–94, native 2–10 per iteration across three runs); they do not establish a cache bottleneck. Invalid L1 load counts are omitted.

4. What changes when N grows?

At five bodies, Barnes-Hut is **3.92× slower**: tree setup/traversal does not pay off for ten pairs. Development NumPy and struct-of-arrays variants also lose at this size; FMM was not measured. Unrolled source grows as $O(N^2)$, so the shipped generator falls back to a rolled loop above 20,000 pairs. Detailed code-size experiments are WSL2/CPython 3.10.21 development evidence, distinct from the VM sweep below (dev/nbody/FINDINGS.md).

The VM force-evaluation sweep below includes a rebuilt Barnes-Hut tree at each evaluation ($\theta = 0.5$, best of seven). It evaluates an extended workload, not the required $N = 5$ benchmark.

N	Direct	BH	BH/direct	Median rel. error
5	0.020 ms	0.077 ms	3.92×	0
100	4.746 ms	8.410 ms	1.77×	1.6e-5
300	42.977 ms	42.489 ms	0.99×	3.3e-5
800	313.489 ms	158.846 ms	0.51×	8.7e-5
1,600	1,254.660 ms	382.471 ms	0.30×	1.8e-4

The crossover is near $N = 300$ for this distribution; median acceleration error is not a worst-case or trajectory bound. A separate VM native sweep gives 21–22× over rolled Python at $N = 100$ –3,200, using approximately equal pair-update counts (results/bigN_sweep.txt).

5. Hardware acceleration: current design

`grape_pipeline` retains state on the device and executes a complete `advance(dt, n)` request. Force computation suits a scheduled datapath, while velocity accumulation must respect dependencies between pairs sharing a body. Steps remain serial.

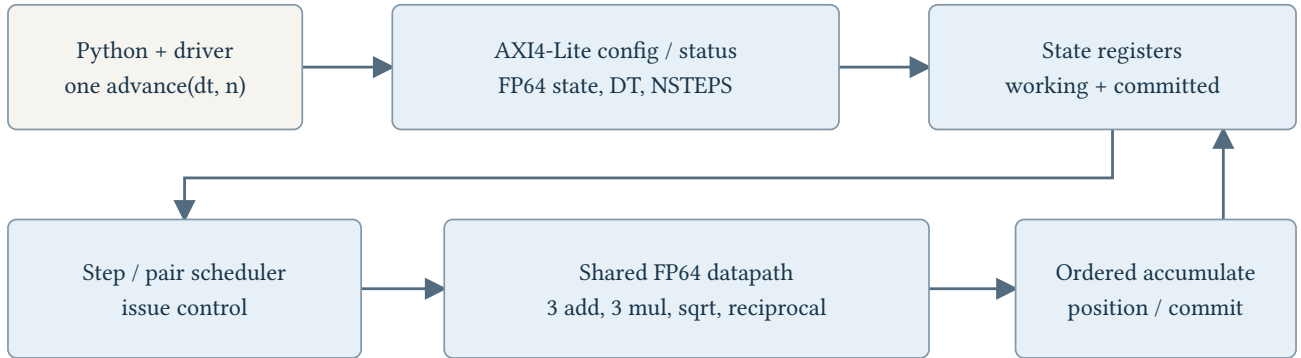


Figure 4: Current datapath and software boundary, summarized from the approved architecture. Multiplication and accumulation round separately; there is no FMA unit.

Datapath. Three FP64 add/subtract units, three multipliers, one square-root unit and one reciprocal unit share a scheduled 290-operation step. The square root uses radix-4 iteration; the reciprocal uses a ROM seed and refined fixed-point arithmetic. An ordered accumulation sequencer resolves body-component dependencies before positions are updated and committed.

Interface. A 32-bit AXI4-Lite slave exposes FP64 body state and DT, 32-bit NSTEPS, pair configuration, control/status, counters and IRQ. Software loads state and parameters, starts one request, waits, then reads state. There is no per-step DMA or Python call. A register-level driver model implements this protocol and is checked against the register map. It is not a deployed device driver or evidence of a physical CPU/accelerator run.

Dependencies determine the schedule

Pairs (0, 1) and (0, 2) can form their distance terms from the same step's positions, but both update body 0's velocity. Its x lane must compute $u0a = u0 - dx01 * b1m$, then $u0b = u0a - dx02 * b2m$; issuing both against the old $u0$ would lose an update. Regrouping them into one subtraction may also change FP64 rounding. Different body-component lanes can progress independently; a scoreboard and ordered issue logic wait for each lane's preceding result. Positions update only after their required velocity contributions, and the next step uses the committed positions.

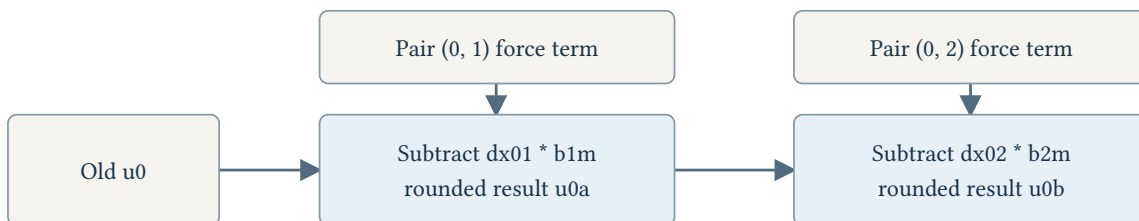


Figure 5: Illustrative dependency chain for body 0's x velocity. Force terms may be prepared independently, while the two rounded velocity updates remain ordered.

Verification and physical cost

Evidence	Result and scope
Full-benchmark RTL simulation	124 cycles/step; 2,480,000 cycles over 20,000 steps; zero mismatches against the hardware arithmetic model
Recorded regression	9/9 tests on Verilator and Icarus after the prefix rewrite; numerical checks against stock use tolerances
Architectural clock target	50 MHz; 49.6 ms compute time if achieved
Latest post-CTS timing	≈ 19.46 MHz, typical corner, <code>grape_prefix2</code> (2026-09-14); ≈ 127.4 ms computed execution time
Physical completion / power	No completed routed sign-off or GDS; no workload power result

The schedule predicted 123–127 cycles/step; RTL measured 124. Clock-tree synthesis (CTS) adds the clock distribution network, so post-CTS static timing includes a placed clock tree, but is still a preliminary timing estimate rather than measured silicon performance. The 19.46 MHz estimate derives from a 150 ns constraint and +98.6212 ns setup slack. The design has not demonstrated the 50 MHz target.

Recorded architecture	Mapped cell area	Cycles/step
One-wide accumulation, one-port RF	2.938 mm ²	162
Three-wide accumulation, three-port RF	4.075 mm ²	124

The three-wide design spends 38.7% more mapped area to reduce cycles/step by 23.5%. These sky130 synthesis areas describe the recorded pre-prefix design points; they are not a new area measurement for `grape_prefix2`. The subsequent prefix rewrite replaces a roughly 60-deep issue-selection scan with balanced trees, preserving 124 cycles/step while improving the post-CTS frequency estimate from 11.15 to 19.46 MHz. The critical path moved from the accumulate-adder selection to the integrate-multiplier operand path. More parallelism can reduce cycles while adding selection, forwarding and wiring cost that limits the clock.

End-to-end estimate. Use $T_{\text{new}} = T_{\text{remaining}} + \text{cycles} / \text{frequency} + T_{\text{interface}}$. At 19.46 MHz, compute alone is 127.4 ms: its ratios against 231.20 ms stock and 143.13 ms optimized Python are upper bounds of about 1.81 $\times$ and 1.12 $\times$ before residual work and overhead. For illustration, retaining 1–5% of stock runtime gives about 1.65–1.78 $\times$ before interface cost. That residual range is a sensitivity assumption, not a matched phase measurement. The register protocol estimates roughly 150 AXI-Lite transactions per invocation; real host software and interconnect latency remain unmeasured. Even the 50 MHz target’s 49.6 ms is slower than the 9.53 ms native software. No energy-efficiency advantage follows without workload power and complete system measurements.

The hardware’s square-root/reciprocal expression differs from stock `pow`. RTL is checked against its own arithmetic model. Comparison with stock targets relative energy error $\leq 1\text{e-}12$ and per-body position/velocity errors $\leq 2\text{e-}9 / 5\text{e-}11$ at completion. These tolerance targets are distinct from the software tier’s observed exact equality.

6. Conclusion

Specializing a fixed schedule cuts Python runtime by 38.1% while preserving the tested state and energy exactly. Native execution removes more interpreter work through the same coarse interface. The hardware design implements that boundary, but its benefit over native software requires substantially better clock/latency results than demonstrated so far. The measured cycle/area trade-off and preliminary physical timing explain why a fast schedule alone is insufficient.

Evidence: `hw/grape_pipeline/docs/uarch.md`, `prd.md`, `testplan.md`, `ppa.md` (including the 2026-09-14 addendum), and `integration.md`. Reproduction and supporting profiles: **report_appendix.pdf**. Repository: [HWSW_Project](#).